

05.-Creacion-de-Imágenes

En esta actividad aprenderemos a **crear imágenes Docker a partir de aplicaciones**, utilizando dos tecnologías diferentes:

- **Python + Flask** → aplicación to-do-list-v1
- **Node.js + CSS** → aplicación to-do-list-v2

Aplicación To-Do List con Python

Para el primer ejercicio utilizaremos una aplicación desarrollada en Python, correspondiente a una lista de tareas.

Construcción de la imagen

`docker build -t to-do-list-v1:1.0 .`

```
ubuntu:26.04 226011b31c8 1c00b 45.30B
jezuz@DESKTOP-NQ62FUL:/mnt/c/Users/Jesus/Desktop/Galaxy Training/Docker/Examen-Final/05.-Creacion-de-Imagenes/App-Python$ docker build -t to-do-list-v1:1.0 .
[+] Building 2.0s (13/13) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.28kB
=> resolve image config for docker-image://docker.io/docker/dockerfile:1
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:ecfaec9ed5d810b56388c508f4121597bfbba70d41a6dfcee4d8cad5f295fc32
=> => resolve docker.io/docker/dockerfile:1@sha256:ecfaec9ed5d810b56388c508f4121597bfbba70d41a6dfcee4d8cad5f295fc32
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 381B
=> [1/6] FROM docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
=> => resolve docker.io/library/python:3.12-slim@sha256:78387bc3881b8273120a12ebe6c1ab22b018ccc2c9adf565ae1ac9b536e184ea
=> [internal] load build context
=> => transferring context: 63B
=> CACHED [2/6] WORKDIR /app
=> CACHED [3/6] COPY requirements.txt
=> CACHED [4/6] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/6] COPY .
=> CACHED [6/6] RUN groupadd -r appgroup && useradd -r -g appgroup appuser && chown -R appuser:appgroup /app
```

Creamos del contenedor

A partir de la imagen creada ejecutamos el contenedor

`docker container run -d \`

`> -p 5000:5000 \`

`> --name CONTENEDOR-TO-DO-LIST-v1 \`

`> to-do-list-v1:1.0`

`c5ba446b2c8f7b3e2ed0872bc3195878c2c0cf8ab19fb50b9bd9118d93712761`

```
jezuz@DESKTOP-NQ62FUL:/mnt/c/Users/Jesus/Desktop/Galaxy Training/Docker/Examen-Final/05.-Creacion-de-Imagenes/App-Python$ docker container run -d \
> -p 5000:5000 \
> --name CONTENEDOR-TO-DO-LIST-v1 \
> to-do-list-v1:1.0
c5ba446b2c8f7b3e2ed0872bc3195878c2c0cf8ab19fb50b9bd9118d93712761
jezuz@DESKTOP-NQ62FUL:/mnt/c/Users/Jesus/Desktop/Galaxy Training/Docker/Examen-Final/05.-Creacion-de-Imagenes/App-Python$
```

Validación de la aplicación

Accedemos desde el navegador:

```
localhost:5000/tasks
localhost:5000/tasks
Aplicaciones | Controlador de Ver... | Login Autónoma Vir... | Ingles | Diccio... | Configuración de cá... | movistar | [LAB] Introducció...
Dar formato al texto
[{"created_at":"2026-09-08T15:23:25.040464+00:00","done":false,"id":1,"title":"Estudiar Docker"}, {"created_at":"2026-09-08T15:23:59.08941+00:00","done":false,"id":2,"title":"Enviar el examen de Docker"}, {"created_at":"2026-09-08T15:24:14.519183+00:00","done":false,"id":3,"title":"Enviar el examen de Docker"}]
```

Aplicación To-Do List con Node.js

Para el segundo ejercicio utilizaremos una aplicación desarrollada con **Node.js**, incorporando CSS para mejorar su presentación.

El objetivo es repetir el mismo proceso anterior, pero utilizando una aplicación construida con una tecnología diferente.

Construcción de la imagen

`docker build -t to-do-list-v2:1.0 .`

```
jezuz@DESKTOP-NQ62FUL:/mnt/c/Users/Jesus/Desktop/Galaxy Training/Docker/Examen-Final/05.-Creacion-de-Imagenes/Node$ docker build -t tarjeta-notas-v2:1.0 .
[+] Building 5.8s (15/15) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 502B
=> resolve image config for docker-image://docker.io/docker/dockerfile:1
=> [auth] docker/dockerfile:pull token for registry-1.docker.io
=> CACHED docker-image://docker.io/docker/dockerfile:1@sha256:ecfaec9edd810b56388c588f4121597bfbba70d41a6dfee4d8cad5f295fc32
=> => resolve docker.io/docker/dockerfile:1@sha256:ecfaec9edd810b56388c588f4121597bfbba70d41a6dfee4d8cad5f295fc32
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 236B
=> [1/6] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a8b0d56d59c58a04632e6c0fb8363f609372293
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a8b0d56d59c58a04632e6c0fb8363f609372293
=> [internal] load build context
=> => transferring context: 4.52kB
=> CACHED [2/6] WORKDIR /app
```

Creamos del contenedor

A partir de la imagen creada ejecutamos el contenedor

```
docker container run -d \
> -p 3000:3000 \
> --name CONTENEDOR-TARJETA-NOTAS-V2 \
> tarjeta-notas-v2:1.0
```

```
jezuz@DESKTOP-NQ62FUL:/mnt/c/Users/Jesus/Desktop/Galaxy Training/Docker/Examen-Final/05.-Creacion-de-Imagenes/Node$ docker container run -d \
> -p 3000:3000 \
> --name CONTENEDOR-TARJETA-NOTAS-V2 \
> tarjeta-notas-v2:1.0
a28a26a4196f895637a63ac68525468c024412bab8e0596f3e60ac27ba3328a7
jezuz@DESKTOP-NQ62FUL:/mnt/c/Users/Jesus/Desktop/Galaxy Training/Docker/Examen-Final/05.-Creacion-de-Imagenes/Node$
```

Validación de la aplicación

Accedemos desde el navegador:

